Serviço Público Federal
Ministério da Educação
Fundação Universidade Federal de Mato Grosso do Sul

Faculdade de Computação
Núcleo de Práticas em Engenharia de Software

Documento de Implantação

Carteira Cidadã

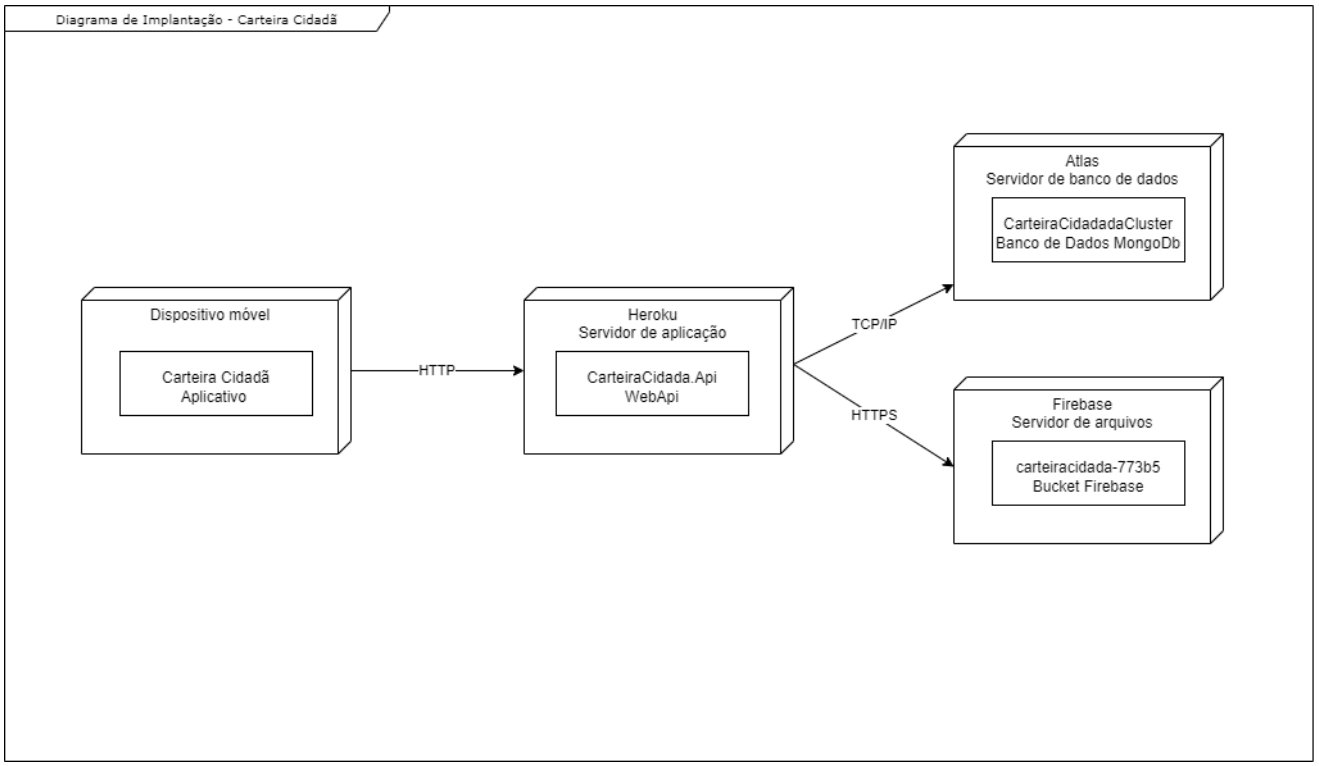
Versão 1

Histórico de Alterações do Documento

Data	Versão	Descrição	Autor
09.06.2022	1.0	Preenchimento da estrutura de pastas do Back-End	Breno
15.06.2022	2.0	Preenchimento da tabela de variáveis de ambiente do Back-End	Breno
22.06.2022	3.0	Adicionando diagrama de implantação e link para o manual de implantação do Back-end	Breno

1. Diagrama de Implantação

Nesta seção será apresentado o diagrama de implantação dos subsistemas que compõem o projeto.



2. Estrutura do código fonte

Nesta seção serão apresentadas a estrutura de pastas e arquivos de todos os subsistemas que compõem o diagrama de implantação.

2.1. Estrutura do Back-End

Nesta subseção será apresentada a estrutura de pastas e arquivos do Back-End presente no diagrama de implantação.

- **/src:** Pasta que contém toda a codebase
  - **CarteiraCidada.sln:** Arquivo de solução .NET que representa todo o projeto do back-end;
  - **Dockerfile:** Arquivo utilizado para gerar a imagem docker (que é usada no deploy Heroku) da solução final;
  - **/CarteiraCidada.Api:** Pasta que contém a codebase relacionada à comunicação HTTP e autenticação;

- **/Controllers:** Pasta que contém todas as controladoras da aplicação;
- **/Authorization:** Pasta que contém arquivos que lidam com autorização e autenticação, como validação de AppSecret e Middleware JWT;
- **/ErrorHandler:** Pasta que contém o middleware que trata erros lançados na aplicação, retornando diferentes HTTP Status Code dependendo do erro;
- **/wwwroot:** Pasta que contém as fotos que os clientes da aplicação enviam;
- **/Properties:** Pasta que contém arquivos de configuração para executar a aplicação no Visual Studio;
- **appsettings.json:** Arquivo JSON que contém as variáveis de configuração da aplicação para produção;
- **appsettings.Development.json:** Arquivo JSON que contém as variáveis de configuração de aplicação para desenvolvimento;
- **CarteiraCidada.Api.csproj:** Arquivo de projeto C# contendo as definições de pacotes e referências entre os projetos e opções de compilação dessa solução;
- **Program.cs:** Entry point da aplicação, contendo o método Main();
- **Startup.cs:** Contém a classe que inicializa as configurações da aplicação (ex: middlewares) quando ela é iniciada**;**

◆ **/CarteiraCidada.Application:** Pasta que contém a codebase relacionada aos casos de uso da aplicação;

- **/Exceptions:** Pasta que contém todas as exceções personalizadas criadas pelos desenvolvedores que podem ser lançadas nessa camada;
- **/Extensions:** Pasta que contém as configurações de injeção de dependências dessa camada;
- **/Shared:** Pasta que contém arquivos comuns/compartilhados entre os demais arquivos desta camada (com exceção das Exceptions);
- **/UseCases:** Pasta que contém todos os casos de uso da aplicação, assim como suas validações de Input;
- **CarteiraCidada.Application.csproj:** Arquivo de projeto C# contendo as definições de pacotes e referências entre os projetos e opções de compilação dessa solução;

- **/CarteiraCidada.Domain:** Pasta que contém a codebase relacionada às classes de domínio da aplicação;
    - **/Auth:** Pasta que contém arquivos relacionados à autenticação e autorização no nível de domínio;
    - **/Entities:** Pasta que contém todas as entidades manipuladas pelos casos de uso;
    - **/Models:** Pasta que contém os arquivos que são comuns entre os outros contextos de domínio;
    - **CarteiraCidada.Domain.csproj:** Arquivo de projeto C# contendo as definições de pacotes e referências entre os projetos e opções de compilação dessa solução;
- **/CarteiraCidada.Infra:** Pasta que contém a codebase relacionada à conexão e manipulação do banco de dados;
    - **/Collections:** Pasta que contém as classes que representam as MongoDB Collections na aplicação;
    - **/Extensions:** Pasta que contém as configurações de injeção de dependências dessa camada;
    - **/Repositories:** Pasta que contém as implementações dos repositórios declarados na camada de domínio;
    - **CarteiraCidada.Domain.csproj:** Arquivo de projeto C# contendo as definições de pacotes e referências entre os projetos e opções de compilação dessa solução;

➔ **/test:** Pasta que contém todos os testes automatizados;

- **/CarteiraCidada.UnitTests:** Pasta que contém todos os testes unitários da aplicação;

2.2. Estrutura da aplicação mobile

O projeto foi organizado, principalmente, em 4 diretórios: core, features, utils e zen_components.

O diretório core contém classes necessárias para a execução do aplicativo, como o arquivo de definição de rotas, gerenciador de dependências, arquivo de definição do cliente http, entre outros.

O diretório utils contém classes que oferecem ações/recursos utilizados em diversas partes da aplicação. Dentre as ações, destaca-se: conversões de tipos, adição de métodos em tipos pré-existentes na linguagem, entre outros.

O diretório zen_components contém os componentes reutilizáveis de UI do projeto, assim como os estilos utilizados.

O diretório features contém as funcionalidades do projeto, além de um diretório compartilhado de data. Cada feature é composta pelos diretórios data e apresentação. O diretório data contém os datasources, as models de response que são retornadas pela API, além dos repositories. Já a camada de presentation contém, basicamente, as views e os controllers.

```
├──core
├──features
│   ├──auth
│   │   ├──data
│   │   │   ├──api
│   │   │   ├──entity
│   │   │   ├──local
│   │   │   └──repository
│   │   └──presentation
│   │       ├──signin
│   │       └──signup
│   ├──data
│   │   └──entity
│   ├──feed
│   │   ├──data
│   │   │   ├──api
│   │   │   ├──entity
│   │   │   └──repository
│   │   └──presentation
│   ├──home
│   │   └──presentation
│   ├──profile
│   │   ├──data
│   │   │   ├──api
│   │   │   ├──entity
│   │   │   └──repository
│   │   └──presentation
│   │       └──components
│   ├──social_action
│   │   ├──data
│   │   │   ├──api
│   │   │   ├──entity
│   │   │   └──repository
│   │   └──presentation
│   │       └──create
│   ├──social_action_application
│   │   └──data
│   │       ├──api
│   │       ├──entity
│   │       └──repository
│   └──voluntary_action
│       ├──data
│       │   ├──entity
│       │   └──repository
│       └──presentation
│           └──mappers
├──utils
│   ├──extensions
│   └──validators
└──zen_components
```

3. Instalação e Execução

Nesta seção serão apresentados os procedimentos necessários para a instalação e execução do projeto.

3.1. Back-End

Link do Manual de Implantação do Back-end:
Manual de implantação Back-end.pdf

3.1.1. Ambiente de Desenvolvimento

3.1.1.1. Variáveis de ambiente

Variável	Valor para produção
ConnectionString	*Substituir pelo valor gerado usando o manual de implantação*
DatabaseName	*Substituir pelo valor gerado usando o manual de implantação*
TokenSecret	FCB932192C8D10928DE3D5BBAD2CD9E7C5D58036132A9F664E665D2F4B93EF15
FirebaseAPIKey	*Substituir pelo valor gerado usando o manual de implantação*
FirebaseBucket	*Substituir pelo valor gerado usando o manual de implantação*
FirebaseUserEmail	*Substituir pelo valor gerado usando o manual de implantação*
FirebaseUserPassword	*Substituir pelo valor gerado usando o manual de implantação*

3.1.2. Ambiente de Produção

3.1.2.1. Variáveis de ambiente

Variável	Valor para produção
ConnectionString	*Substituir pelo valor gerado usando o manual de implantação*
DatabaseName	*Substituir pelo valor gerado usando o manual de implantação*
TokenSecret	FCB932192C8D10928DE3D5BBAD2CD9E7C5D58036132A9F664E665D2F4B93EF15
FirebaseAPIKey	*Substituir pelo valor gerado usando o manual de implantação*
FirebaseBucket	*Substituir pelo valor gerado usando o manual de implantação*
FirebaseUserEmail	*Substituir pelo valor gerado usando o manual de implantação*
FirebaseUserPassword	*Substituir pelo valor gerado usando o manual de implantação*

3.2. Aplicação Mobile - Android

O Aplicativo foi desenvolvido utilizando o framework Flutter na versão 2.10.5, e a linguagem Dart em sua versão 2.16.2. Tanto para preparação do Ambiente do Desenvolvimento como para geração do artefato de release, é necessário além da instalação do framework, a instalação do Android Studio ou do Android SDK. O passo a passo será detalhado nas subseções seguintes;

3.2.1. Ambiente de Desenvolvimento

Partiremos do ponto em que o código do sistema já foi clonado do repositório no GitHub, e já está na máquina local. O primeiro passo a ser executado é a instalação do Flutter. Antes de iniciar com a preparação certifique-se que seu sistema operacional atende os requisitos descritos na documentação do Flutter.

3.2.1.1. Instalação e configuração do Flutter (Windows)

Obter o Flutter SDK:

Primeiramente, iremos obter o Flutter SDK. Para isso, temos dois caminhos. Fazer o download do arquivo .zip ou clonar o repositório do Flutter. Seguiremos o guia através da primeira opção.
Para obter o Flutter SDK, acesse esse link e baixe o arquivo correspondente a versão 2.10.2 na listagem de SDK do *Stable Channel*.

Stable channel (Windows)

Select from the following scrollable list:

3.0.0	x64	ee4e09c	5/11/2022	2.17.0
2.10.5	x64	5464c5b	4/18/2022	2.16.2
2.10.4	x64	c860cba	3/28/2022	2.16.2
2.10.3	x64	7e9793d	3/2/2022	2.16.1
2.10.2	x64	097d331	2/19/2022	2.16.1
2.10.1	x64	db747aa	2/9/2022	2.16.1
2.10.0	x64	5f105a6	2/3/2022	2.16.0

Com o SDK baixado, vá para a raiz do diretório “C:\” e crie uma pasta com o nome “src”. Em seguida, extraia o .zip do Flutter SDK na pasta recém criada.

Atualizar variáveis de Ambiente:
Para atualizar suas variáveis de ambiente, e s com isso ser capaz de executar comandos Flutter pelo terminal no Windows, siga os seguintes passos:

- Na barra de pesquisa do menu **Iniciar**, insira 'env' e selecione *Editar variáveis de ambiente para sua conta* .
- Em **variáveis do usuário**, verifique se há uma entrada chamada **Path**. Se a entrada existir, anexe o caminho completo para **‘flutter\bin’** (usar **ponto e vírgula** como separador dos valores existentes).
- Se a entrada não existir, crie uma nova variável de usuário nomeada **Path** com o caminho completo para **‘flutter\bin’** como seu valor.
- Você precisa fechar e reabrir todas as janelas de console existentes para que essas alterações entrem em vigor.

3.2.1.2. Checando ambiente e instalação

Para validar se os passos anteriores foram executados corretamente, abra seu terminal e execute o comando ‘flutter run’. O resultado provável será algo como a imagem abaixo:

```
[-] Android toolchain - develop for Android devices
    • Android SDK at D:\Android\sdk
    ✗ Android SDK is missing command line tools; download from https://goo.gl/XxQghQ
    • Try re-installing or updating your Android SDK,
      visit https://docs.flutter.dev/setup/#android-setup for detailed instructions.
```

Esse comando serve para checar o ambiente, e exibe um diagnóstico sobre o status da instalação do Flutter mostrando as pendências. Neste caso, a ausência do Android SDK é um problema que merece atenção e será resolvido na próxima subseção.

3.2.1.3. Instalação do Android Studio e Android SDK

Para instalar a IDE e o Android SDK basta seguir os seguintes passos:

- Baixe e instale o Android Studio.
- Inicie o Android Studio e vá até o 'Assistente de configuração do Android Studio'. Isso instala o Android SDK mais recente, as ferramentas de linha de comando do Android SDK e as ferramentas de compilação do Android SDK, que são exigidas pelo Flutter ao desenvolver para Android.
- Execute **flutter doctor** para confirmar que o Flutter localizou sua instalação do Android Studio. Se o Flutter não conseguir localizá-lo, execute **flutter config --android-studio-dir <directory>** para definir o diretório no qual o Android Studio está instalado.

3.2.1.4. Configurando dispositivo Android

Para se preparar para executar e testar seu aplicativo Flutter em um dispositivo Android, você precisa de um dispositivo Android executando o Android 4.1 (API de nível 16) ou superior.

- Ative as **opções do desenvolvedor** e a **depuração USB** no seu dispositivo. Instruções detalhadas estão disponíveis na documentação do Android .
- Somente Windows: instale o driver USB do Google .
- Usando um cabo USB, conecte seu telefone ao computador. Se solicitado em seu dispositivo, autorize seu computador a acessar seu dispositivo.

- No terminal, execute o comando **flutter devices** para verificar se o Flutter reconhece seu dispositivo Android conectado. Por padrão, o Flutter usa a versão do Android SDK em que sua ferramenta **adb** está baseada. Se você quiser que o Flutter use uma instalação diferente do Android SDK, você deve definir a **ANDROID_SDK_ROOT** variável de ambiente para esse diretório de instalação.

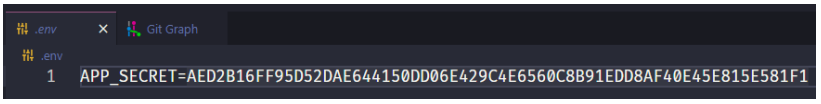
3.2.1.5. Aceitando licenças do Android

Antes de poder usar o Flutter, você deve concordar com as licenças da plataforma Android SDK. Esta etapa deve ser feita após a instalação das ferramentas listadas acima.

- Abra seu terminal e execute o comando a seguir para começar a assinar licenças: **flutter doctor --android-licenses.**
- Revise os termos de cada licença cuidadosamente antes de concordar com eles.
- Quando terminar de concordar com as licenças, execute o flutter doctor novamente para confirmar que você está pronto para usar o Flutter.

3.2.1.6. Definição das variáveis de ambiente do projeto

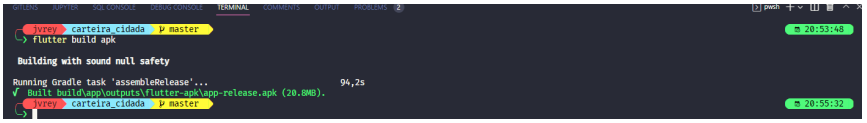
Na raiz do projeto, crie um arquivo ".env" com o seguinte conteúdo (sem aspas):
"APP_SECRET=AED2B16FF95D52DAE644150DD06E429C4E6560C8B91EDD8AF40E45E815E581F1"
O resultado será semelhante a imagem abaixo:



3.2.1.7. Rodando a aplicação

Com um dispositivo ou um emulador conectado ao seu computador, será necessário executar dois comandos na raiz do projeto:

- O primeiro é o **flutter pub get** para obter os pacotes utilizados no projeto;
- E em seguida, execute o comando **flutter run** para rodar a aplicação.

3.2.2. Gerando artefato de release (.apk)

Para gerar o artefato de release, será necessário ter cumprido as etapas descritas na seção **3.2.1**. E em seguida, basta executar, na raiz do projeto, o comando **flutter build apk**, ou **flutter build appbundle** para gerar o .apk de release ou o appbundle, respectivamente. O resultado será semelhante ao exibido abaixo: